



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



ASIGNATURA METODOLOGÍAS DE ANALÍTICA DE DATOS

Actividad Autónoma 3

Nombre de la actividad: KDD aplicado y comparación metodológica

Unidad 2: Enfoques clásicos de la analítica

Tema 1: Knowledge Discovery in Databases (KDD)



FACULTAD DE
Ingeniería

Nombres:

Fecha:

Carrera:

Periodo académico:

Semestre:



Objetivo de la actividad: Distinguir con precisión los enfoques descriptivo, predictivo y prescriptivo, justificando su uso en problemas reales y definiendo para cada uno pregunta, datos de entrada, técnicas, salida/entregable y métrica de éxito

Recursos o temas que debe haber estudiado antes de hacer la actividad:

Conceptos básicos de analítica de datos

- Fases de la metodología Knowledge Discovery in Databases (KDD)
- Herramientas y técnicas más utilizadas en KDD
- Ejemplo de aplicación de KDD en un dataset real.
- Comparación de KDD con otras metodologías de minería de datos.

Formato de entrega: PDF

Instrucciones:

- Redacte claramente las respuestas solicitadas
- Siga los pasos de la actividad autónoma en secuencia estricta
- Incluya una bibliografía con al menos 5 referencias académicas en APA 7.
- Nombrar el archivo de la siguiente manera:
Apellido_Nombre_Grupo_X.

Contenido

1. Explique con sus propias palabras qué es KDD y describa el propósito de cada una de sus cinco fases (selección, preprocesamiento, transformación, minería, interpretación). Indique además por qué este flujo es necesario en proyectos de analítica.
2. Clasifique cada actividad en la fase de KDD que corresponda y justifique en 2–3 líneas:
 - a. Crear una bitácora de calidad, detectar duplicados y estandarizar formatos de fecha.
 - b. Generar variables de recencia, frecuencia y valor (RFM) y aplicar una reducción de dimensionalidad (PCA).
 - c. Entrenar y comparar árboles de decisión, regresión logística y gradient boosting con validación cruzada.
 - d. Definir la ventana temporal, las fuentes autorizadas y los criterios de inclusión/exclusión de registros.
 - e. Elaborar un dashboard de hallazgos, explicar errores y proponer umbrales operativos.

- f. Imputar faltantes con reglas de negocio y KNN-imputer y documentar supuestos.
3. Mencione e investigue seis técnicas comunes en un proyecto de KDD y asigne cada una a su fase y a una herramienta posible. Describa en una línea el objetivo y un entregable típico.
- Pistas:** muestreo estratificado; reglas de validación; one-hot encoding; scaling; PCA; selección por information gain; clustering k-means; reglas de asociación; matriz de confusión/ROC; informe de interpretación.
4. Ordene cronológicamente y asigne la fase de KDD a cada elemento. Justifique en 1 frase.
- a) Definir la ventana “t-6 a t-1 meses” y listar tablas/variables autorizadas.
 - b) Quitar outliers por IQR y normalizar montos.
 - c) Construir variables de “canasta” y lags semanales.
 - d) Comparar modelos y seleccionar el mejor por ROC-AUC y F1.
 - e) Presentar un mapa de segmentos y reglas de acción para marketing.
 - f) Unir ventas internas con clima y calendario de promociones.

Realice el siguiente ejercicio:

Problema: La ANT desea entender y anticipar la emisión de licencias de conducir por provincia, categoría y priorizar oficinas.

Objetivo: Aplicar KDD sobre el archivo “ant_datos_licencias_2022_mayo_hoja.csv” de la página de Datos Abiertos del Ecuador para:

- **Limpieza y documentación de calidad de datos agregados:** El estudiante limpiará el archivo agregado (provincia×categoría×mes), estandarizará encabezados y tipos, imputará vacíos coherentemente y documentará la calidad del dato (tamaño, % de faltantes, consistencia de totales frente a suma de categorías, y hallazgos relevantes).
- **Indicadores descriptivos por provincia, categoría y mes (cuando aplique):** El estudiante construirá indicadores de volumen y mezcla: totales por provincia, distribución por clase/categoría y, si la hoja lo permite, totales por mes. Se incluirán visualizaciones como barras comparativas y mapas de calor (absoluto y en proporciones) para sustentar los hallazgos.
- **Modelo de priorización provincial con variables de mezcla:** El estudiante desarrollará un modelo sencillo a nivel provincia (p.

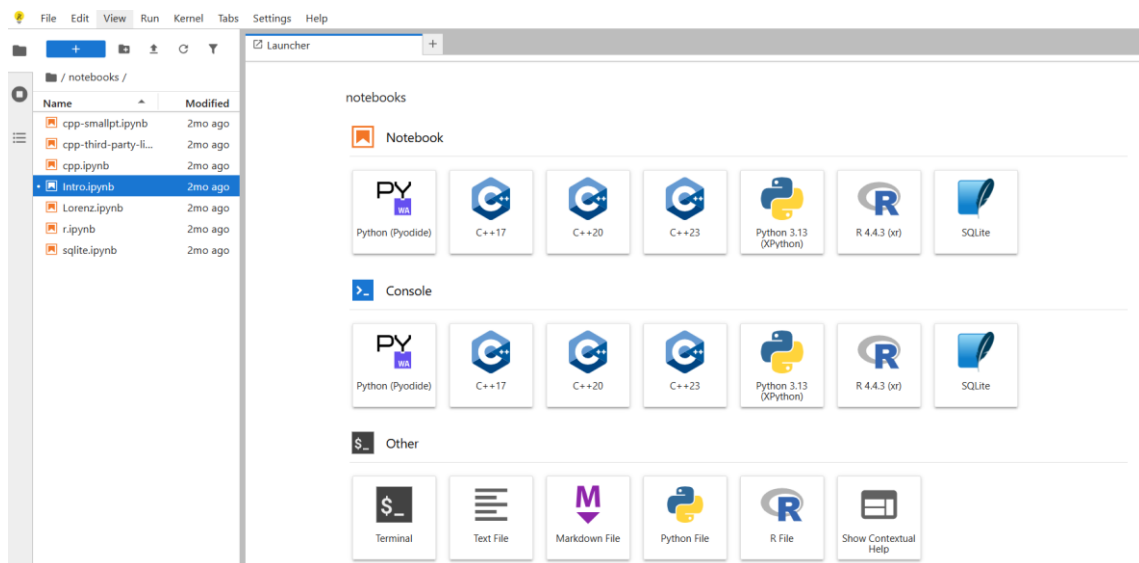
ej., clasificación Top vs. no-Top por cuartil o share profesional proxy) utilizando como insumos las proporciones por categoría (prop_*). Se reportarán métricas (Accuracy, Precision, Recall, F1 y, cuando corresponda, ROC-AUC) para justificar el uso del modelo como herramienta de priorización táctica.

- **Traducción de resultados a decisiones operativas:** Con base en los indicadores y el modelo, el estudiante propondrá acciones de focalización: priorización de provincias de mayor carga, refuerzo por categorías con lift alto (especialización relativa), y lineamientos operativos (p. ej., dimensionamiento de recursos y focos de campaña). Se señalarán supuestos y límites derivados de trabajar con datos agregados.

Indicador Clave de Desempeño KPI

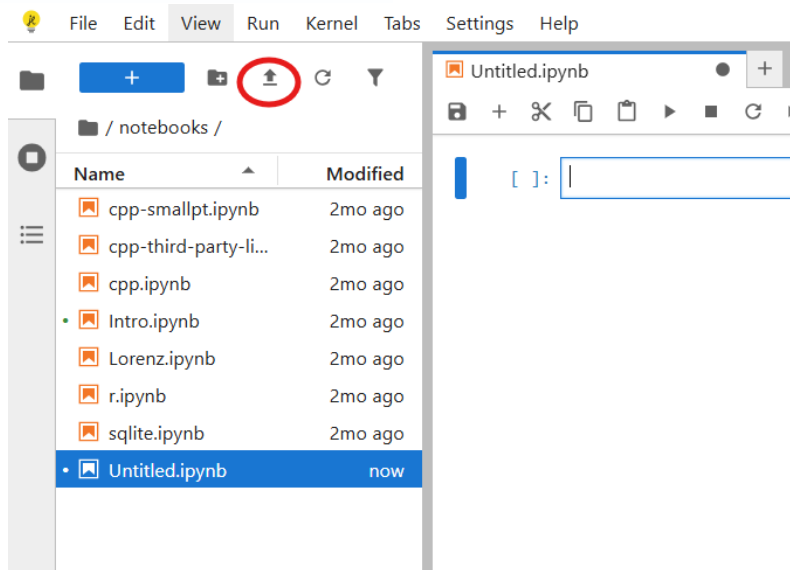
- a. Lift/variación de demanda por provincia y categoría;
- b. Accuracy/F1 del modelo y uso para focalización operativa.

Ingresa a: <https://jupyter.org/try-jupyter/lab/>



Seleccione Pyodide dado que sirve para ejecutar Python directamente en el navegador.

Suba el archivo “ant-datos-licencias-2022-mayo.xlsx” con la flecha indicada en la imagen.



FASE 0 — Preparación

```
import sys, numpy as np, pandas as pd, matplotlib.pyplot as plt

from pathlib import Path

np.random.seed(42)

pd.set_option("display.max_columns", 200)

def std_name(c):

    return (str(c).strip().replace("\n", "").replace("\r", "")

            .lower().replace(" ", "_").replace("-", "_")

            .replace("á", "a").replace("é", "e").replace("í", "i")

            .replace("ó", "o").replace("ú", "u").replace("ñ", "n"))

def normaliza_txt(x):

    s = str(x).strip().upper()

    for a,b in (("Á", "A"), ("É", "E"), ("Í", "I"), ("Ó", "O"), ("Ú", "U"), ("Ü", "U"), ("Ñ", "N")): s = s.replace(a,b)

    return s

ON_WEB = (sys.platform == "emscripten") # JupyterLite/Pyodide

print("Entorno:", "Web (sin pip)" if ON_WEB else sys.platform)
```

FASE 1 — Selección y normalización (CSV)

```
import numpy as np, pandas as pd

from pathlib import Path
```

```
# 1) Ruta del CSV exportado desde la hoja del Excel

RUTA_CSV = "ant_datos_licencias_2022_mayo_hoja.csv" # <-- ajusta al nombre real

assert Path(RUTA_CSV).exists(), f'No se encontró {RUTA_CSV}. Sube el CSV correcto.'

# --- utilidades ---

def std_name(c):

    return (str(c).strip().replace("\n", "").replace("\r", "")

            .lower().replace(" ", "").replace("-", "_")

            .replace("á", "a").replace("é", "e").replace("í", "i")

            .replace("ó", "o").replace("ú", "u").replace("ñ", "n"))

# 2) Leer todo como texto, con separador ';' (no asumir encabezado)

raw = pd.read_csv(RUTA_CSV, sep=";", header=None, dtype=str, engine="python", encoding="utf-8")

# 3) Detectar automáticamente la fila que contiene los encabezados reales

TOKENS = {"A", "A1", "B", "C", "C1", "D", "E", "F", "G", "TOTAL GENERAL", "TOTAL_GENERAL", "TOTALGENERAL"}

def score_header_row(row):

    vals = [str(x).strip().upper() for x in row.tolist()]

    hits = sum(v in TOKENS for v in vals)

    nstr = sum(v != "" for v in vals)

    return hits*1000 + nstr # prioriza fila con tokens esperados

hdr_idx = raw.apply(score_header_row, axis=1).idxmax()

header_vals = [str(x).strip() for x in raw.iloc[hdr_idx].tolist()]

if header_vals and (header_vals[0] == "" or header_vals[0].lower().startswith("unnamed")):

    header_vals[0] = "unidad"

cols_std = [std_name(c) for c in header_vals]
```

4) Construir df_crudo con ese encabezado (sin perder filas)

```
df_crudo = raw.iloc[hdr_idx+1:].copy()
```

```
df_crudo.columns = cols_std
```

```
# limpiar texto SOLO en columnas objeto (sin FutureWarning)
```

```
df_crudo = df_crudo.apply(lambda s: s.map(lambda x: str(x).strip() if pd.notna(x) else x)
                           if s.dtype == "object" else s)
```

```
# asegurar la primera columna como 'unidad'
```

```
if df_crudo.columns[0] != "unidad":
```

```
    df_crudo = df_crudo.rename(columns={df_crudo.columns[0]: "unidad"})
```

```
# quedarnos con columnas relevantes si existen
```

```
cols_esperadas = ["unidad", "a", "a1", "b", "c", "c1", "d", "e", "f", "g", "total_general"]
```

```
df_crudo = df_crudo.loc[:, df_crudo.notna().any(axis=0)]
```

```
df_crudo = df_crudo[[c for c in cols_esperadas if c in df_crudo.columns]]
```

```
# 5) Limpieza numérica segura: quitar miles, coma→punto, '-'→NaN y convertir
```

```
clases_cols = [c for c in ["a", "a1", "b", "c", "c1", "d", "e", "f", "g"] if c in df_crudo.columns]
```

```
num_cols = clases_cols + (["total_general"] if "total_general" in df_crudo.columns else [])
```

```
df_crudo = df_crudo.replace({"-": np.nan, "—": np.nan, ",": np.nan})
```

```
for c in num_cols:
```

```
    df_crudo[c] = (df_crudo[c].astype(str)
```

```
                  .str.replace(r"^[^d.\-]", "", regex=True) # deja solo dígitos y separadores
```

```
                  .str.replace(".", "", regex=False)        # quita punto de miles
```

```
                  .str.replace(",", ".", regex=False))      # coma → punto
```

```
    df_crudo[c] = pd.to_numeric(df_crudo[c], errors="coerce")
```

```
# 6) Sanearamiento: clases NaN → 0 y total_general consistente con suma de clases
```



```
if clases_cols:

    df_crudo[clases_cols] = df_crudo[clases_cols].fillna(0)

    suma_clases = df_crudo[clases_cols].sum(axis=1)

    if "total_general" in df_crudo.columns:

        df_crudo["total_general"] = df_crudo["total_general"].fillna(suma_clases)

        dif = (df_crudo["total_general"] - suma_clases).abs()

        df_crudo.loc[dif > 0.5, "total_general"] = suma_clases

    else:

        df_crudo["total_general"] = suma_clases

    # opcional: castear a int para reporte

    df_crudo[clases_cols + ["total_general"]] = df_crudo[clases_cols + ["total_general"]].astype(int,
errors="ignore")

# 7) Salida de control

print(f"FASE 1 OK → filas: {df_crudo.shape[0]} | columnas: {df_crudo.shape[1]}")

print("Columnas:", list(df_crudo.columns))

display(df_crudo.head(12))
```

FASE 2 — Extraer provincias (24)

```
import pandas as pd

import numpy as np

assert 'df_crudo' in globals(), "Primero ejecuta la Fase 1 (df_crudo)."

# 1) utilidades

def normaliza_txt(x: str) -> str:

    s = str(x).strip().upper()

    repl = (("Á", "A"), ("É", "E"), ("Í", "I"), ("Ó", "O"), ("Ú", "U"), ("Ü", "U"), ("Ñ", "N"))

    for a,b in repl: s = s.replace(a,b)

    s = " ".join(s.split()) # compactar espacios
```




return s

```
MESES = { "ENERO","FEBRERO","MARZO","ABRIL","MAYO","JUNIO",  
  
          "JULIO","AGOSTO","SEPTIEMBRE","OCTUBRE","NOVIEMBRE","DICIEMBRE"  
  
        }  
  
PALABRAS_TIPO = ["DUPLICADO","REIMPRES","RECATEG","PRIMERA VEZ","RENOVAC","EMISION"]  
  
# Catálogo oficial (normalizado) de provincias  
  
PROVINCIAS = {  
  
    "AZUAY","BOLIVAR","CANAR","CARCHI","COTOPAXI","CHIMBORAZO","EL  
ORO","ESMERALDAS","GUAYAS",  
  
    "IMBABURA","LOJA","LOS RIOS","MANABI","MORONA SANTIAGO","NAPO","PASTAZA","PICHINCHA",  
  
    "SANTA ELENA","SANTO DOMINGO DE LOS TSACHILAS","SUCUMBIOS","TUNGURAHUA","ZAMORA  
CHINCHIPE",  
  
    "GALAPAGOS","ORELLANA"  
  
}  
  
# 2) normalizar etiqueta y contar columnas numéricas por fila  
  
clases_cols = [c for c in ["a","a1","b","c","c1","d","e","f","g","total_general"] if c in df_crudo.columns]  
  
dfN = df_crudo.copy()  
  
dfN["unidad_norm"] = dfN["unidad"].map(normaliza_txt)  
  
dfN["_n_num"] = dfN[clases_cols].notna().sum(axis=1)  
  
# 3) reglas de clasificación  
  
mask_cat = dfN["unidad_norm"].isin(PROVINCIAS)  
  
mask_mes = dfN["unidad_norm"].isin(MESES)  
  
mask_tipo = False  
  
for k in PALABRAS_TIPO:  
  
    mask_tipo = mask_tipo | dfN["unidad_norm"].str.contains(k, na=False)
```

Heurística extra: filas alfabéticas cortas, con ≥ 3 columnas numéricas, que no son mes ni tipo

```
mask_heur = (  
    ~mask_mes & ~mask_tipo &  
    dfN["unidad_norm"].str.match(r"^[A-Z\s\.\,]+$", na=False) &  
    (dfN["unidad_norm"].str.len() <= 30) &  
    (dfN["_n_num"] >= 3)  
)  
  
mask_prov = mask_cat | mask_heur  
  
df_prov = dfN.loc[mask_prov, ["unidad"] + clases_cols].copy()
```

4) quedarnos con UNA fila por provincia (a veces los reportes repiten subtotales)

Usamos la primera aparición con mayor 'total_general' si hay duplicados.

```
if "total_general" in df_prov.columns:  
    df_prov["_tg"] = df_prov["total_general"].fillna(0)  
  
    df_prov = (df_prov  
        .assign(unidad_norm=df_prov["unidad"].map(normaliza_txt))  
        .sort_values(["unidad_norm", "_tg"], ascending=[True, False])  
        .drop_duplicates("unidad_norm", keep="first")  
        .drop(columns=["unidad_norm", "_tg"])  
        .reset_index(drop=True))  
else:  
    df_prov = (df_prov  
        .assign(unidad_norm=df_prov["unidad"].map(normaliza_txt))  
        .drop_duplicates("unidad_norm", keep="first")  
        .drop(columns=["unidad_norm"])  
        .reset_index(drop=True))
```

5) verificación

```
prov_detectadas = sorted(df_prov["unidad"].map(normaliza_txt).unique().tolist())
```

```
faltantes = sorted(list(PROVINCIAS - set(prov_detectadas)))
```

```
print(f"Provincias detectadas: {len(prov_detectadas)}")
```

```
print("Listado:", prov_detectadas)
```

```
if faltantes:
```

```
    print("\nFaltantes (según catálogo):", faltantes)
```

6) ordenar y mostrar tabla final de provincias

```
df_prov = df_prov.sort_values("unidad").reset_index(drop=True)
```

```
display(df_prov.head(15))
```

```
display(df_prov.tail(15))
```

```
print("Forma final df_prov:", df_prov.shape)
```

```
import pandas as pd
```

```
import numpy as np
```

```
assert 'df_crudo' in globals(), "Primero ejecuta la Fase 1 (df_crudo)."
```

1) utilidades

```
def normaliza_txt(x: str) -> str:
```

```
    s = str(x).strip().upper()
```

```
    repl = (("Á","A"),("É","E"),("Í","I"),("Ó","O"),("Ú","U"),("Ü","U"),("Ñ","N"))
```

```
    for a,b in repl: s = s.replace(a,b)
```

```
    s = " ".join(s.split()) # compactar espacios
```

```
    return s
```

```
MESES = {
```

```
    "ENERO","FEBRERO","MARZO","ABRIL","MAYO","JUNIO",
```



```
"JULIO","AGOSTO","SEPTIEMBRE","OCTUBRE","NOVIEMBRE","DICIEMBRE"

}

PALABRAS_TIPO = ["DUPLICADO","REIMPRES","RECATEG","PRIMERA VEZ","RENOVAC","EMISION"]

# Catálogo oficial (normalizado) de provincias

PROVINCIAS = {

    "AZUAY","BOLIVAR","CANAR","CARCHI","COTOPAXI","CHIMBORAZO","EL
    ORO","ESMERALDAS","GUAYAS",

    "IMBABURA","LOJA","LOS RIOS","MANABI","MORONA SANTIAGO","NAPO","PASTAZA","PICHINCHA",

    "SANTA ELENA","SANTO DOMINGO DE LOS TSACHILAS","SUCUMBIOS","TUNGURAHUA","ZAMORA
    CHINCHIPE",

    "GALAPAGOS","ORELLANA"

}

# 2) normalizar etiqueta y contar columnas numéricas por fila

clases_cols = [c for c in ["a","a1","b","c","c1","d","e","f","g","total_general"] if c in df_crudo.columns]

dfN = df_crudo.copy()

dfN["unidad_norm"] = dfN["unidad"].map(normaliza_txt)

dfN["_n_num"] = dfN[clases_cols].notna().sum(axis=1)

# 3) reglas de clasificación

mask_cat = dfN["unidad_norm"].isin(PROVINCIAS)

mask_mes = dfN["unidad_norm"].isin(MESES)

mask_tipo = False

for k in PALABRAS_TIPO:

    mask_tipo = mask_tipo | dfN["unidad_norm"].str.contains(k, na=False)

# Heurística extra: filas alfabéticas cortas, con ≥3 columnas numéricas, que no son mes ni tipo

mask_heur = (

    ~mask_mes & ~mask_tipo &
```

```
dfN["unidad_norm"].str.match(r"^[A-Z\s\.,]+$", na=False) &

(dfN["unidad_norm"].str.len() <= 30) &

(dfN["_n_num"] >= 3)

)

mask_prov = mask_cat | mask_heur

df_prov = dfN.loc[mask_prov, ["unidad"] + clases_cols].copy()

# 4) quedarnos con UNA fila por provincia (a veces los reportes repiten subtotales)

# Usamos la primera aparición con mayor 'total_general' si hay duplicados.

if "total_general" in df_prov.columns:

    df_prov["_tg"] = df_prov["total_general"].fillna(0)

    df_prov = (df_prov

        .assign(unidad_norm=df_prov["unidad"].map(normaliza_txt))

        .sort_values(["unidad_norm", "_tg"], ascending=[True, False])

        .drop_duplicates("unidad_norm", keep="first")

        .drop(columns=["unidad_norm", "_tg"])

        .reset_index(drop=True))

else:

    df_prov = (df_prov

        .assign(unidad_norm=df_prov["unidad"].map(normaliza_txt))

        .drop_duplicates("unidad_norm", keep="first")

        .drop(columns=["unidad_norm"])

        .reset_index(drop=True))

# 5) verificación

prov_detectadas = sorted(df_prov["unidad"].map(normaliza_txt).unique().tolist())

faltantes = sorted(list(PROVINCIAS - set(prov_detectadas)))
```

```
print(f"Provincias detectadas: {len(prov_detectadas)}")

print("Listado:", prov_detectadas)

if faltantes:

    print("\nFaltantes (según catálogo):", faltantes)


# 6) ordenar y mostrar tabla final de provincias

df_prov = df_prov.sort_values("unidad").reset_index(drop=True)

display(df_prov.head(15))

display(df_prov.tail(15))

print("Forma final df_prov:", df_prov.shape)
```

FASE 3 — Transformación

```
import numpy as np, pandas as pd

assert 'df_prov' in globals(), "Antes debe ejecutarse la Fase 2 (o 2.5) para construir df_prov."

# 1) Columnas de clases presentes en df_prov

clases_presentes = [c for c in ["a","a1","b","c","c1","d","e","f","g"] if c in df_prov.columns]

assert len(clases_presentes) > 0, "No se encontraron columnas de clases en df_prov."


# 2) Copia de trabajo y saneo: clases NaN → 0 (evita perder provincias al pasar a largo)

dfp = df_prov.copy()

dfp[clases_presentes] = dfp[clases_presentes].fillna(0)


# 3) Provincias → formato largo (sin dropna); luego convertir a int

prov_long = dfp.melt(

    id_vars=["unidad"],

    value_vars=clases_presentes,

    var_name="clase",

    value_name="emisiones"

)

# asegurar numérico
```

```
prov_long["emisiones"] = pd.to_numeric(prov_long["emisiones"], errors="coerce").fillna(0).astype(int)
```

```
print("Chequeo rápido:")
```

```
print(" - Provincias en df_prov:", dfp.shape[0])
```

```
print(" - Filas en prov_long (debería ser 24 × n° de clases):", prov_long.shape[0])
```

```
display(prov_long.head(12))
```

```
# 4) Total por provincia (suma de clases)
```

```
tot_calc = prov_long.groupby("unidad",  
as_index=False)["emisiones"].sum().rename(columns={"emisiones": "total_calc"})
```

```
# Integrar total_general si existe; si difiere mucho, priorizar suma de clases
```

```
df_prov2 = dfp.merge(tot_calc, on="unidad", how="left")
```

```
if "total_general" in df_prov2.columns:
```

```
    dif = (df_prov2["total_general"].fillna(0) - df_prov2["total_calc"].fillna(0)).abs()
```

```
    df_prov2["_total"] = np.where(dif > 0.5, df_prov2["total_calc"], df_prov2["total_general"])
```

```
else:
```

```
    df_prov2["_total"] = df_prov2["total_calc"]
```

```
# 5) Proporciones por clase (mezcla) — cada fila suma ≈ 1.0
```

```
suma_clases = df_prov2[clases_presentes].sum(axis=1)
```

```
suma_clases = suma_clases.replace(0, np.nan) # evita división por cero
```

```
for c in clases_presentes:
```

```
    df_prov2[f"prop_{c}"] = (df_prov2[c] / suma_clases).round(6)
```

```
# 6) Ordenar por total y mostrar
```

```
df_prov2 = df_prov2.sort_values("_total", ascending=False).reset_index(drop=True)
```

```
cols_mostrar = ["unidad", "_total"] + clases_presentes + [f"prop_{c}" for c in clases_presentes]
```

```
print("\nResumen df_prov2 (debe haber 24 provincias):", df_prov2.shape)

display(df_prov2[cols_mostrar].head(12))

# 7) Verificación final (cuenta y suma de proporciones)

print("\nProvincias en df_prov2:", df_prov2.shape[0])

print("Suma de proporciones por provincia (esperado ≈ 1.0):")

display(df_prov2[[f"prop_{c}" for c in clases_presentes]].sum(axis=1).round(3).head(10))

# (Opcional) exportar insumos

# prov_long.to_csv("provincia_clase_largo.csv", index=False)

# df_prov2.to_csv("provincia_totales_y_mezcla.csv", index=False)
```

FASE 4 — Clasificación Top vs. no-Top (autosuficiente)

```
import numpy as np, pandas as pd

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate

from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,

                             roc_auc_score, confusion_matrix, classification_report,

                             make_scorer)

from sklearn.ensemble import RandomForestClassifier

assert 'df_prov2' in globals(), "Ejecuta primero la Fase 3 para construir df_prov2."

# 0) Asegurar que existan columnas de mezcla prop_*; si no, crearlas desde las clases

mix_cols = [c for c in df_prov2.columns if c.startswith("prop_")]

if not mix_cols:

    clases_presentes = [c for c in ["a","a1","b","c","c1","d","e","f","g"] if c in df_prov2.columns]

    assert clases_presentes, "df_prov2 no tiene columnas de clases ni prop_*."

    suma = df_prov2[clases_presentes].sum(axis=1).replace(0, np.nan)

    for c in clases_presentes:

        df_prov2[f"prop_{c}"] = (df_prov2[c] / suma).round(6)
```

```
mix_cols = [f"prop_{c}" for c in clases_presentes]

# 1) Etiqueta Top (Q3) con salvaguarda (mín. 25% o 3 provincias)

score = df_prov2["_total"]

q3 = score.quantile(0.75)

y = (score >= q3).astype(int)

min_pos = max(3, int(np.ceil(len(df_prov2) * 0.25)))

if y.sum() < min_pos:

    idx_top = score.sort_values(ascending=False).head(min_pos).index

    y = pd.Series(0, index=score.index); y.loc[idx_top] = 1

    print(f"[Ajuste] Pocos Top con Q3. Se usa Top-N = {min_pos}.")

# 2) Datos y partición

X = df_prov2[mix_cols].fillna(0)

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.30, random_state=42, stratify=y)

# 3) Modelo

clf = RandomForestClassifier(n_estimators=600, class_weight="balanced",

                             random_state=42, n_jobs=-1)

clf.fit(Xtr, ytr)

# 4) Predicción con umbral por ranking (para evitar "sin positivos")

proba_te = clf.predict_proba(Xte)[:, 1]

k_exp = max(1, int(round(ytr.mean() * len(yte)))) # nº esperado de positivos en test

order = np.argsort(-proba_te)

pred_rank = np.zeros_like(yte); pred_rank[order[:k_exp]] = 1

acc = accuracy_score(yte, pred_rank)
```

```
prec = precision_score(yte, pred_rank, zero_division=0)

rec = recall_score(yte, pred_rank, zero_division=0)

f1 = f1_score(yte, pred_rank, zero_division=0)

auc = roc_auc_score(yte, proba_te) if len(np.unique(yte))==2 else float("nan")


print("== Test ==")

print(f'Accuracy: {acc:.3f} | Precision: {prec:.3f} | Recall: {rec:.3f} | F1: {f1:.3f}')

print("ROC-AUC:", f"{auc:.3f}" if np.isfinite(auc) else "n/d") print("Matriz de
confusión:\n", confusion_matrix(yte, pred_rank)) print("Reporte:\n",
classification_report(yte, pred_rank, digits=3, zero_division=0))


# 5) CV estratificada

n_splits = max(2, min(5, int(y.value_counts().min())))

cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)

scorers = {

    "accuracy": make_scorer(accuracy_score),

    "precision": make_scorer(precision_score, zero_division=0),

    "recall": make_scorer(recall_score, zero_division=0),

    "f1": make_scorer(f1_score, zero_division=0),

}

res = cross_validate(clf, X, y, cv=cv, scoring=scorers, n_jobs=-1)

print("\n== CV estratificada ==")

display(pd.DataFrame({k: res[f'test_{k}'] for k in scorers}).agg(["mean", "std"]).round(3))


# 6) Importancias y ranking global

imp = pd.Series(clf.feature_importances_, index=mix_cols).sort_values(ascending=False).round(3)

print("\nImportancia de variables (prop_*):")

display(imp)
```

```
proba_all = clf.predict_proba(X)[: , 1]

df_rank = df_prov2.assign(prob_top=proba_all).sort_values("prob_top", ascending=False)

df_rank["pred_top_rank"] = 0

topN_global = max(1, int(round(y.mean() * len(df_rank))))

df_rank.iloc[:topN_global, df_rank.columns.get_loc("pred_top_rank")] = 1

print("\nTop potencial (primeras 15 filas):")

display(df_rank[["unidad", "_total", "prob_top", "pred_top_rank"] + mix_cols].head(15))
```

FASE 5 — Assess: gráficos + KPI + export

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt

from pathlib import Path

from textwrap import wrap

assert 'df_prov2' in globals() and 'prov_long' in globals(), "Ejecuta la Fase 3 primero."

# -----

# A. AUXILIARES / ENTRADAS

# -----

mix_cols = [c for c in df_prov2.columns if c.startswith("prop_")]

clases_cols = [c for c in ["a","a1","b","c","c1","d","e","f","g"] if c in df_prov2.columns]

# Top real (Q3) con salvaguarda ≈25% o al menos 3

score = df_prov2["_total"]

q3 = score.quantile(0.75)

es_top = (score >= q3).astype(int)

min_pos = max(3, int(np.ceil(len(df_prov2)*0.25)))

if es_top.sum() < min_pos:

    idx_top = score.sort_values(ascending=False).head(min_pos).index

    es_top = pd.Series(0, index=score.index); es_top.loc[idx_top] = 1
```



```
# Probabilidades del modelo (si vienen de Fase 4C)

prob_top = None

if 'df_rank' in globals() and {"unidad","prob_top"}.issubset(df_rank.columns):

    prob_top = df_rank.set_index("unidad")["prob_top"].reindex(df_prov2["unidad"]).values


# Función para "envolver" etiquetas largas

def wrap_labels(ax, width=12):

    labs = [ "\n".join(wrap(str(t.get_text()), width)) for t in ax.get_xticklabels() ]

    ax.set_xticklabels(labs)


# -----

# B. GRÁFICOS DESCRIPTIVOS

# -----


# B1) Barras: total por provincia fig, ax =

plt.subplots(figsize=(12,5))

(df_prov2.sort_values("_total", ascending=False)

    .plot(x="unidad", y="_total", kind="bar", ax=ax,

        title="Emisiones por provincia (Total)"))

ax.set_xlabel("Provincia"); ax.set_ylabel("Emisiones")

ax.tick_params(axis='x', labelrotation=30, labelsz=9)

wrap_labels(ax, 14)

plt.subplots_adjust(bottom=0.28, top=0.90, left=0.08, right=0.98)

plt.show()


# B2) Heatmap: absoluto (provincia × clase)

pivot_abs = (prov_long.pivot_table(index="unidad", columns="clase",

    values="emisiones", aggfunc="sum", fill_value=0)

    .reindex(df_prov2["unidad"]))
```

```
fig, ax = plt.subplots(figsize=(12,7))

im = ax.imshow(pivot_abs.values, aspect="auto")

fig.colorbar(im, ax=ax, label="Emisiones")

ax.set_title("Heatmap: provincia × clase (absoluto)")

ax.set_yticks(range(len(pivot_abs.index))); ax.set_yticklabels(pivot_abs.index)

ax.set_xticks(range(len(pivot_abs.columns))); ax.set_xticklabels(pivot_abs.columns)

plt.tight_layout(); plt.show()

# B3) Heatmap: proporciones (mezcla por provincia)

fila_suma = pivot_abs.sum(axis=1).replace(0, np.nan)

pivot_prop = (pivot_abs.div(fila_suma, axis=0)).fillna(0)

fig, ax = plt.subplots(figsize=(12,7))

im = ax.imshow(pivot_prop.values, aspect="auto")

fig.colorbar(im, ax=ax, label="Proporción")

ax.set_title("Heatmap: provincia × clase (proporciones)")

ax.set_yticks(range(len(pivot_prop.index))); ax.set_yticklabels(pivot_prop.index)

ax.set_xticks(range(len(pivot_prop.columns))); ax.set_xticklabels(pivot_prop.columns)

plt.tight_layout(); plt.show()

# -----

# C. KPI (a) — Lift y Variación por provincia×categoría

# -----

# Definiciones:

# - prop_pais[c] = participación nacional de la clase c

# - prop_prov[p,c] = participación de la clase c dentro de la provincia p

# - LIFT[p,c] = prop_prov[p,c] / prop_pais[c]

# - VAR[p,c] = prop_prov[p,c] - prop_pais[c] (puntos porcentuales)

prop_pais = (pivot_abs.sum(axis=0) / pivot_abs.values.sum()) # vector por clase

lift = pivot_prop.div(prop_pais, axis=1).replace([np.inf,-np.inf], np.nan).fillna(0.0)
```

```
var_pp = (pivot_prop - prop_pais)
```

```
# Tabla de "oportunidad" ponderando por tamaño provincial (total)
```

```
op_score = (lift * df_prov2.set_index("unidad")["_total"]).reindex(lift.index).values.reshape(-1,1))
```

```
top_lift = (op_score.stack()
```

```
    .reset_index().rename(columns={"level_0":"unidad","level_1":"clase",0:"lift_pond"})
```

```
    .sort_values("lift_pond", ascending=False))
```

```
# Mostrar TOP 15 oportunidades por lift ponderado
```

```
aux_prop_prov = pivot_prop.stack().rename("prop_prov").reset_index()
```

```
aux_prop_pais = prop_pais.rename("prop_pais").reset_index().rename(columns={"index":"clase"})
```

```
top15 = (top_lift.head(15)
```

```
    .merge(aux_prop_prov, on=["unidad","clase"]))
```

```
    .merge(aux_prop_pais, on="clase"))
```

```
top15["lift"] = (top15["prop_prov"]/top15["prop_pais"]).round(2)
```

```
top15["var_pp"] = (top15["prop_prov"]-top15["prop_pais"]).round(3)
```

```
print("KPI (a) — Top 15 provincia×clase por Lift ponderado (prioridad operativa):")
```

```
display(top15[["unidad","clase","lift","var_pp","lift_pond"]])
```

```
# Gráfico: Top 10 lift ponderado fig, ax =
```

```
plt.subplots(figsize=(11,4)) top_lift.head(10).plot(x="unidad",
```

```
y="lift_pond", kind="bar", ax=ax,
```

```
    title="Top 10 oportunidades (Lift ponderado)")
```

```
ax.set_xlabel("Provincia"); ax.set_ylabel("Lift ponderado")
```

```
ax.tick_params(axis='x', labelrotation=30, labelsz=9); wrap_labels(ax, 14)
```

```
plt.subplots_adjust(bottom=0.28, top=0.90, left=0.08, right=0.98)
```

```
plt.show()
```



```
# -----  
  
# D. KPI (b) — Métricas del modelo y focalización operativa  
  
# -----  
  
# Si no hay prob_top del paso 4C, entrenamos un modelo simple aquí.  
  
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate  
  
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,  
  
                             roc_auc_score, confusion_matrix, classification_report,  
  
                             make_scorer)  
  
from sklearn.ensemble import RandomForestClassifier  
  
  
if not mix_cols:  
  
    # crear proporciones si hiciera falta  
  
    suma = df_prov2[clases_cols].sum(axis=1).replace(0, np.nan)  
  
    for c in clases_cols:  
  
        df_prov2[f"prop_{c}"] = (df_prov2[c]/suma).round(6)  
  
    mix_cols = [f"prop_{c}" for c in clases_cols]  
  
  
if prob_top is None:  
  
    y = es_top.copy()  
  
    X = df_prov2[mix_cols].fillna(0)  
  
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.30, random_state=42, stratify=y)  
  
    clf = RandomForestClassifier(n_estimators=600, class_weight="balanced",  
  
                               random_state=42, n_jobs=-1)  
  
    clf.fit(Xtr, ytr)  
  
    proba_te = clf.predict_proba(Xte)[:,-1]  
  
    # Umbral por ranking (evita "sin positivos")  
  
    k_exp = max(1, int(round(ytr.mean()*len(yte))))  
  
    order = np.argsort(-proba_te)  
  
    pred_rank = np.zeros_like(yte); pred_rank[order[:k_exp]] = 1
```

```
acc = accuracy_score(yte, pred_rank)

prec = precision_score(yte, pred_rank, zero_division=0)

rec = recall_score(yte, pred_rank, zero_division=0)

f1 = f1_score(yte, pred_rank, zero_division=0)

auc = roc_auc_score(yte, proba_te) if len(np.unique(yte))==2 else float("nan")

print("\nKPI (b) — Métricas del modelo (clasificación Top vs. no-Top)")

print(f'Accuracy: {acc:.3f} | Precision: {prec:.3f} | Recall: {rec:.3f} | F1: {f1:.3f} | ROC-AUC: {auc:.3f}')

print("Matriz de confusión:\n", confusion_matrix(yte, pred_rank))

print("Reporte:\n", classification_report(yte, pred_rank, digits=3, zero_division=0))

# CV estratificada

n_splits = max(2, min(5, int(y.value_counts().min())))

cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)

scorers = {"accuracy": make_scorer(accuracy_score),

          "precision": make_scorer(precision_score, zero_division=0),

          "recall": make_scorer(recall_score, zero_division=0),

          "f1": make_scorer(f1_score, zero_division=0)}

res = cross_validate(clf, X, y, cv=cv, scoring=scorers, n_jobs=-1)

print("\nCV estratificada (media±std):")

display(pd.DataFrame({k:res[f'test_{k}'] for k in scorers}).agg(['mean','std']).round(3))

# Ranking global para focalización

prob_top = clf.predict_proba(X)[:,-1]

# Tabla de focalización: provincias priorizadas por prob_top

df_focus = df_prov2.copy()

df_focus["prob_top"] = prob_top
```




```
df_focus = df_focus.sort_values("prob_top", ascending=False).reset_index(drop=True)
```

```
print("\nFocalización operativa — Top 12 provincias por prob_top:")
```

```
display(df_focus[["unidad", "_total", "prob_top"] + mix_cols].head(12))
```

```
fig, ax = plt.subplots(figsize=(11,4))
```

```
df_focus.head(12).plot(x="unidad", y="prob_top", kind="bar", ax=ax,
```

```
title="Top 12 provincias por probabilidad de ser TOP")
```

```
ax.set_xlabel("Provincia"); ax.set_ylabel("Prob. TOP")
```

```
ax.tick_params(axis='x', labelrotation=30, labelsz=9); wrap_labels(ax, 14)
```

```
plt.subplots_adjust(bottom=0.28, top=0.90, left=0.08, right=0.98)
```

```
plt.show()
```

```
# -----
```

```
# E. EXPORTABLES (CSV/PNG)
```

```
# ----- Path("export").mkdir(exist_ok=True)
```

```
pivot_abs.to_csv("export/heatmap_abs_valores.csv")
```

```
pivot_prop.to_csv("export/heatmap_prop_valores.csv")
```

```
lift.to_csv("export/kpi_lift_provincia_clase.csv")
```

```
var_pp.to_csv("export/kpi_variacion_pp_provincia_clase.csv")
```

```
df_focus.to_csv("export/focalizacion_prob_top.csv", index=False)
```

```
df_prov2.to_csv("export/provincia_totales_y_mezcla.csv", index=False)
```

```
print("\nArchivos generados en ./export:")
```

```
for f in Path("export").iterdir():
```

```
    print("-", f.name)
```

Responder:

- Adjunte las capturas de la ejecución de cada fase
- Explique en al menos 3 líneas el funcionamiento y la utilidad de cada fase del trabajo realizado.

- Se afirma que Manabí y Pichincha deben tener la misma prioridad operativa que Guayas. En desacuerdo, ya que el lift ponderado muestra oportunidades mayores en Guayas para categorías específicas; la priorización debe ser escalonada (Guayas primero y luego Manabí/Pichincha según mezcla), ¿está de acuerdo con lo anterior o los datos reflejan otras conclusiones?

Bibliografía:

- freeCodeCamp. (s. f.). Data Analysis with Python. Recuperado el 18 de septiembre de 2025, de <https://www.freecodecamp.org/learn/data-analysis-with-python>
- Agencia Nacional de Tránsito. (2022). Licencias de conducir profesionales y no profesionales emitidas mensualmente a escala nacional [Conjunto de datos]. Datos Abiertos Ecuador. Recuperado el 26 de septiembre de 2025, de <https://www.datosabiertos.gob.ec/dataset/licencias-de-conducir-profesionales-y-no-profesionales-emitidas-mensualmente-a-escala-nacional>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.
- SAS Institute Inc. (2004). SEMMA methodology: Sample, Explore, Modify, Model, Assess. SAS Institute.
- Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.



Rúbrica de evaluación

Componente de aprendizaje:	Autónomo	X	Contacto con el Docente	
Nombre de la Unidad:	Unidad 2: Enfoques clásicos de la analítica.			
Resultado(s) de aprendizaje:	- Describir los procesos clásicos de descubrimiento de conocimiento en bases de datos (KDD, SEMMA, CRISP-DM) en contextos académicos y profesionales. - Aplicar las metodologías KDD y SEMMA para planificar y ejecutar proyectos de analítica de datos en distintos escenarios de uso.			
Nombre de la Actividad:	KDD aplicado y comparación metodológica.			

Criterios de Evaluación	Escala de Valoración						
	Excelente (10 - 9,1)	Bueno (9 - 8,1)	Satisfactorio (8 - 7)	Necesita mejorar (6,9 - 0,1)	No entrega (0)	Puntaje	Comentarios (SIGEA)
1. Explicación de conceptos de análisis de datos	Define el concepto con claridad y clasifica la mayoría de los casos con justificación adecuada	Define el concepto con claridad y clasifica la mayoría de los casos con justificación adecuada	Explica parcialmente el concepto y clasifica algunos casos con justificación limitada.	Definición es incompleta o confusas, sin justificación adecuada en la clasificación.	No entrega		
2. Identifica el tipo de analítica descriptiva, predictiva o prescriptiva	Identifica con exactitud el tipo de analítica descriptiva, predictiva o prescriptiva y sustenta con métricas y evidencia	Identifica el tipo de analítica descriptiva, predictiva o prescriptiva y clasifica la mayoría de los casos con justificación adecuada.	Explica parcialmente el concepto y clasifica algunos casos con justificación limitada.	No identifica y con justificaciones incompletas o confusas	No entrega		
3. Identifica los ciclos de vida de un proyecto de analítica de datos	Identifica con exactitud al menos seis de las fases del ciclo de vida de un proyecto de analítica de datos	Identifica con exactitud al menos cuatro de las fases del ciclo de vida de un proyecto de analítica de datos.	Identifica con exactitud al menos dos de las fases del ciclo de vida de un proyecto de analítica de datos	Identifica con exactitud a una de seis de las fases del ciclo de vida de un proyecto de analítica de datos	No entrega		
4. Identifica las actividades y el orden a las que pertenecen dentro del ciclo de vida de un proyecto de analítica de datos.	Ordena correctamente y justifica adecuadamente las siete actividades	Ordena correctamente y justifica adecuadamente al menos cinco actividades	Ordena correctamente y justifica adecuadamente al menos tres actividades.	Ordena correctamente y justifica adecuadamente al menos una actividad.	No entrega		
5. Innovación en soluciones (Eje: Innovación / Autonomía)	Sigue las instrucciones y resuelve correctamente el problema planteado y entrega resultados realistas y ajustados a la problemática	Sigue las instrucciones y resuelve correctamente el problema, dando resultados que no se encuentran alineados al problema	Resuelve el problema pero los resultados obtenidos son inexactos y sin coherencia	No propone mejoras	No entrega		

Puntaje total

Los criterios 4 y 5 están alineados a los ejes de formación del Modelo Educativo UNACH "Introspección y Prospectiva" y responden principalmente a dos de los siguientes ejes:

1. Ambiente;
2. Autonomía y adaptabilidad;
3. Comunicación;
4. Desarrollo humano;
5. Ética y valores;
6. Emprendimiento;
7. Inter y multidisciplinariedad;
8. Innovación;
9. Inclusión e interculturalidad;
10. Investigación;
11. Impacto social;
12. Tecnologías.